```
"""
nexus.core.exceptions
=====================

Central exception hierarchy for
NEXUS.

Design rules:
    * Every exception that can cross
a module boundary MUST derive from
      ``NexusError`` so callers can
catch "anything NEXUS raised" without
      accidentally swallowing
unrelated bugs (e.g. a bare
``Exception``).
    * Security-relevant exceptions
(``GuardrailViolation``,
      ``PermissionDeniedError``) are
never silently caught anywhere in the
      codebase — they are always
logged at minimum ``WARNING`` level.
    * Exceptions carry structured
context (a ``details`` dict) rather
than
      only a human-readable message,
so the logging layer can emit
```

```
      machine-parseable audit
records.
"""

from __future__ import annotations

from typing import Any, Mapping


class NexusError(Exception):
    """
    Base class for every exception
raised by NEXUS code.

    Attributes:
        message: Human-readable
description of what went wrong.
        details: Structured, machine-
readable context (e.g. the offending
            action plan, the package
name that triggered a guardrail,
etc.).
            Kept separate from
``message`` so logging/audit code can
attach
            it as structured fields
instead of parsing a free-text
string.
    """
```

```
    def __init__(self, message: str,
*, details: Mapping[str, Any] | None
= None) -> None:
        super().__init__(message)
        self.message = message
        self.details: dict[str, Any]
= dict(details) if details else {}

    def __str__(self) -> str:  #
pragma: no cover - trivial
        if self.details:
            return f"{self.message} |
details={self.details!r}"
        return self.message

    def __repr__(self) -> str:  #
pragma: no cover - trivial
        return f"
{self.__class__.__name__}
({self.message!r}, details=
{self.details!r})"


class ConfigurationError(NexusError):
    """Raised when required
configuration is missing, malformed,
or contradictory."""
```

```
class
ProviderNotRegisteredError(NexusError
):
    """
    Raised by the DI container when
code requests a provider (LLM, TTS,
STT,
    memory backend, etc.) that was
never registered for the current
    configuration/environment.
    """


class GuardrailViolation(NexusError):
    """
    Raised when a proposed action
fails a deterministic safety check
    (blocked package, disallowed
action type, malformed action plan,
etc.).

    This exception must NEVER be
caught and silently discarded. It is
always
    surfaced to the user and always
written to the audit log. See
    ``nexus.security.guardrails``
(introduced in the security module)
```

```
for
    the checks that raise this.
    """


class
ActionExecutionError(NexusError):
    """
    Raised when an *approved* action
plan fails during execution (ADB call
    failed, Appium session dropped,
Playwright navigation timed out,
etc.).

    Distinguished from
``GuardrailViolation``: this is a
runtime/environment
    failure of something that was
permitted to run, not a policy
rejection.
    """


class MemoryError(NexusError):  #
noqa: A001 - intentional shadow
within our own namespace
    """Raised on failures reading
from or writing to short/long-term
memory stores."""
```

```
class VoicePipelineError(NexusError):
    """Raised on failures in the
wake-word, VAD, STT, or TTS pipeline
stages."""


class AgentError(NexusError):
    """Raised when a LangGraph agent
node fails in a way that should halt
the graph."""


class
PermissionDeniedError(NexusError):
    """
    Raised when an action is well-
formed and not on the hard denylist,
but
    the current permission profile
(see ``nexus.security.permissions``,
    introduced later) does not grant
the caller sufficient privilege.
    """
```